Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**

ОТЧЕТ

по лабораторной работе № 11

Название: REST приложение на фреймворке Django
Дисциплина: Языки интернет программирования

Студент ИУ6-64Б Д. О. Романов
(Группа) (Подпись, дата) (И.О. Фамилия)

Преподаватель В. И. Кузнецов
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Цель работы – изучить архитектуру RESTful API на фреймворке Django, освоить библиотеку Django REST Framework для создания API endpoints, научиться работать с форматом JSON, организовать взаимодействие backend и frontend через асинхронные HTTP-запросы, а также автоматизировать документирование API с помощью Swagger (drf-yasg).

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

REST (Representational State Transfer) – это архитектурный стиль проектирования сетевых приложений, основанный на использовании стандартных HTTP-методов (GET, POST, PUT, DELETE) и безсостояний (stateless) взаимодействии между клиентом и сервером. В RESTful API каждый ресурс идентифицируется URI, а операции над ресурсами соответствуют семантике HTTP-методов. Данные передаются в формате JSON или XML. JSON (JavaScript Object Notation) является наиболее распространённым форматом благодаря лаконичности, читаемости и нативной поддержке в JavaScript. Django REST Framework (DRF) – это расширение Django, предоставляющее набор инструментов для быстрого создания REST API: сериализаторы для преобразования объектов моделей в JSON и обратно, декораторы @api_view для определения HTTP-методов endpoint'а, классы ViewSet для стандартных CRUD-операций. Swagger (OpenAPI) – это спецификация и набор инструментов для описания REST API. Библиотека drf-yasg автоматически генерирует интерактивную документацию на основе кода Django, включая список endpoint'ов, ожидаемые параметры, форматы запросов и ответов, а также позволяет тестировать API прямо из браузера. GraphQL представляет собой альтернативный протокол запросов, разработанный Facebook, при котором

клиент самостоятельно формирует структуру ответа, запрашивая только необходимые поля, что снижает объём передаваемых данных по сравнению с REST.

3. ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1. Задание 1. Создание модели Comment

Создана модель Comment с полями time (DateTimeField, auto_now_add=True) и text (CharField, max_length=20). Поле времени автоматически заполняется при создании записи. Поле текста ограничено 20 символами. Выполнены миграции makemigrations и migrate.

Листинг 1 – models.py

```
class Comment(models.Model):
    time = models.DateTimeField(auto_now_add=True)
    text = models.CharField(max_length=20)
    def __str__(self):
        return f'{self.time} - {self.text}'
```

3.2. Задание 2. Создание сериализатора

Создан сериализатор CommentSerializer на основе ModelSerializer из Django REST Framework. Сериализатор автоматически преобразует объекты модели Comment в JSON и обратно.

Листинг 2 – serializers.py

```
from rest_framework import serializers
from .models import Comment
class CommentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Comment
        fields = ['time', 'text']
```

3.3. Задание 3. Создание представлений REST API

Реализованы четыре endpoint'a. Функция home рендерит HTML-шаблон. Функция get_time возвращает текущее время сервера в формате JSON. Функция get_comments извлекает 10 последних записей из таблицы Comment и сериализует их. Функция save_comment принимает JSON с полями time и text, создаёт новую запись через сериализатор и сохраняет её в базу данных.

Листинг 3 – views.py

```
def home(request):
    return render(request, 'comments/home.html')


@api_view(['GET'])
def get_time(request):
    return Response({'time':
timezone.now().isoformat()})
```

```
@api_view(['GET'])
def get_comments(request):
    comments = Comment.objects.order_by('-time')[:10]
    serializer = CommentSerializer(comments, many=True)
    return Response(serializer.data)

@api_view(['POST'])
def save_comment(request):
    serializer = CommentSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)
```

3.4. Задание 4. Настройка маршрутизации URL

Маршрутизация реализована на двух уровнях. В urls.py приложения определены четыре маршрута: корневой для шаблона, time/ для получения времени, comment/ для получения списка комментариев, comment/save/ для сохранения нового комментария. В urls.py проекта включены маршруты приложения и настроен endpoint Swagger.

Листинг 4 – urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('time/', views.get_time, name='time'),
    path('comment/', views.get_comments,
name='comments'),
    path('comment/save/', views.save_comment,
name='save_comment'),]
```

Листинг 5 – urls.py проекта

```
from django.contrib import admin
from django.urls import path, include
from rest_framework import permissions
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title='Comment API',
        default_version='v1',
        description='API для работы с комментариями',),
    public=True,
    permission_classes=(permissions.AllowAny,),)
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('comments.urls')),
    path('swagger/', schema_view.with_ui('swagger',
cache_timeout=0), name='swagger'),]
```

3.5. Задание 5. Настройка проекта и установка зависимостей

Установлены библиотеки djangorestframework и drf-yasg. В settings приложение comments, rest_framework и drf_yasg добавлены в INSTALLED_APPS. Из списка MIDDLEWARE удалены django.middleware.csrf.CsrfViewMiddleware и django.middleware.clickjacking.XFrameOptionsMiddleware для корректной работы REST API.

Листинг 6 - фрагмент settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'drf_yasg',
    'comments']
```

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',

'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    # 'django.middleware.csrf.CsrfViewMiddleware',

'django.contrib.auth.middleware.AuthenticationMiddleware
',

'django.contrib.messages.middleware.MessageMiddleware',
                                                        #
'django.middleware.clickjacking.XFrameOptionsMiddleware'
,
]
```

3.6. Задание 6. Создание HTML-шаблона с JavaScript

Шаблон home.html содержит элемент для отображения времени с кнопкой «Обновить», таблицу для вывода последних 10 комментариев и форму для отправки нового комментария. Взаимодействие с сервером реализовано через Fetch API без перезагрузки страницы. При отправке формы сначала запрашивается текущее время, затем данные отправляются методом POST на endpoint /comment/save/.

Листинг 7 – home.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Комментарии</title>
</head>
<body>
    <h1>Время: <span id="time"></span> <button
onclick="updateTime()">Обновить</button></h1>
    <table id="table" border="1">
        <tr><th>Время</th><th>Текст</th></tr>
    </table>
    <form onsubmit="sendComment(event)">
        <input type="text" id="text" maxlength="20">
        <button type="submit">Отправить</button>
    </form>

    <script>
    async function updateTime() {
        const r = await fetch('/time/');
        const d = await r.json();
        document.getElementById('time').textContent =
d.time;}
```

```
    async function loadComments() {
        const r = await fetch('/comment/');
        const d = await r.json();
        const t = document.getElementById('table');
        t.innerHTML = '<tr><th>Время</th><th>Текст</
th></tr>';
        d.forEach(c => {
            t.innerHTML += `<tr><td>${c.time}</td><td>$
{c.text}</td></tr>`;
        });
    }

    async function sendComment(e) {
        e.preventDefault();
        await updateTime();
        const time =
document.getElementById('time').textContent;
        const text =
document.getElementById('text').value;
        await fetch('/comment/save/', {
            method: 'POST',
            headers: {'Content-Type': 'application/
json'},
            body: JSON.stringify({time, text})
        });
```

3.7. Проверка работы приложения

Проект запущен командой python manage.py runserver. Страница / отображает время, таблицу комментариев и форму. Кнопка «Обновить» обновляет время без перезагрузки страницы. Endpoint /time/ возвращает JSON с текущим временем. Endpoint /comment/ возвращает список последних 10 записей. Endpoint /comment/save/ принимает POST-запрос и сохраняет данные. Страница /swagger/ отображает автоматически сгенерированную документацию API.

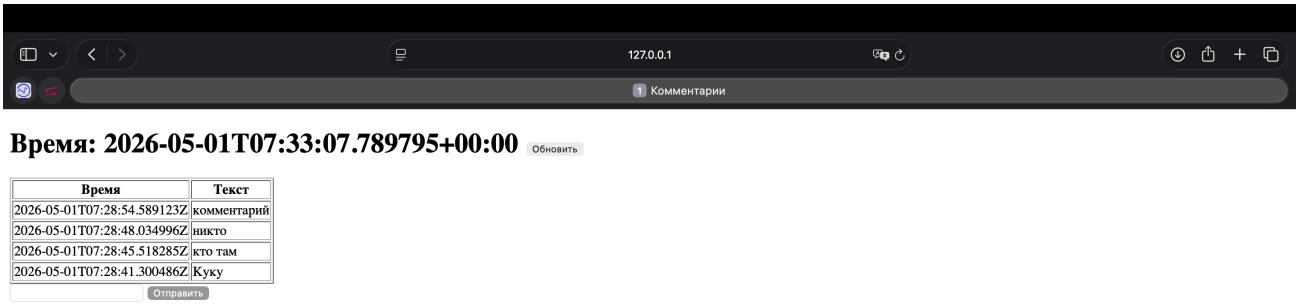
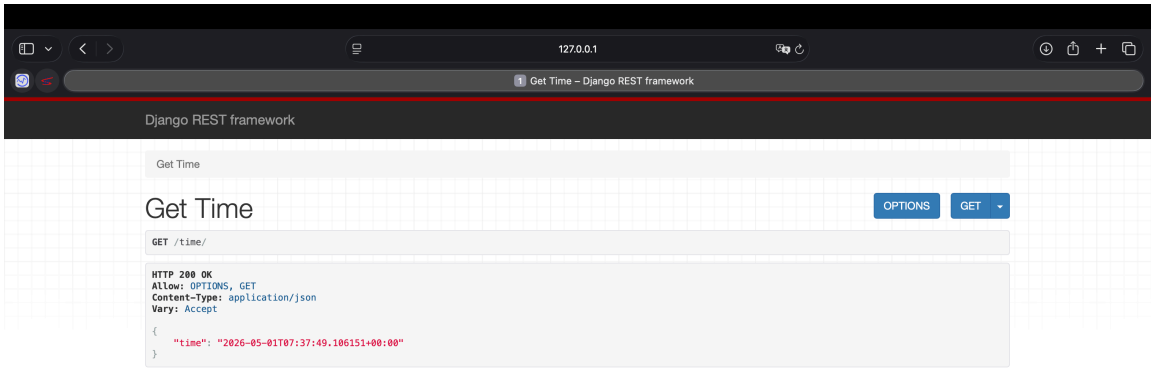


Рисунок 1 – Главная страница



\

Рисунок 2 – Ответ GET /time/

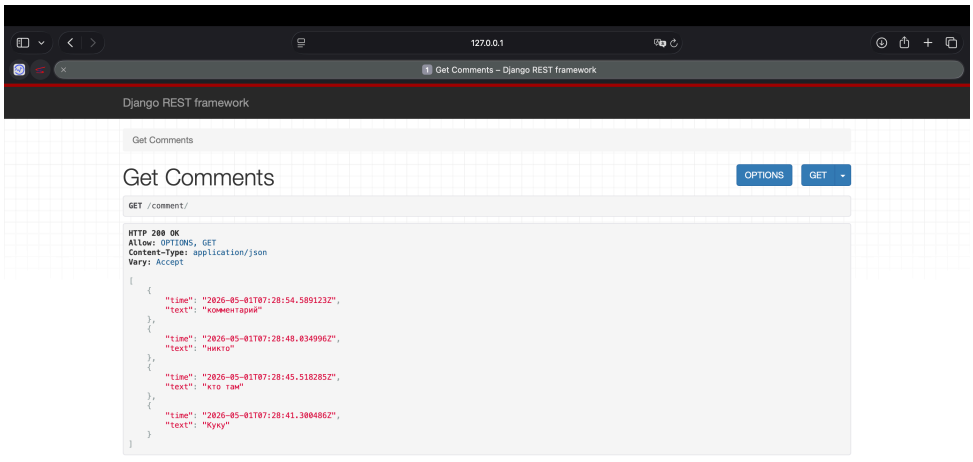


Рисунок 3 – Ответ GET /comment/

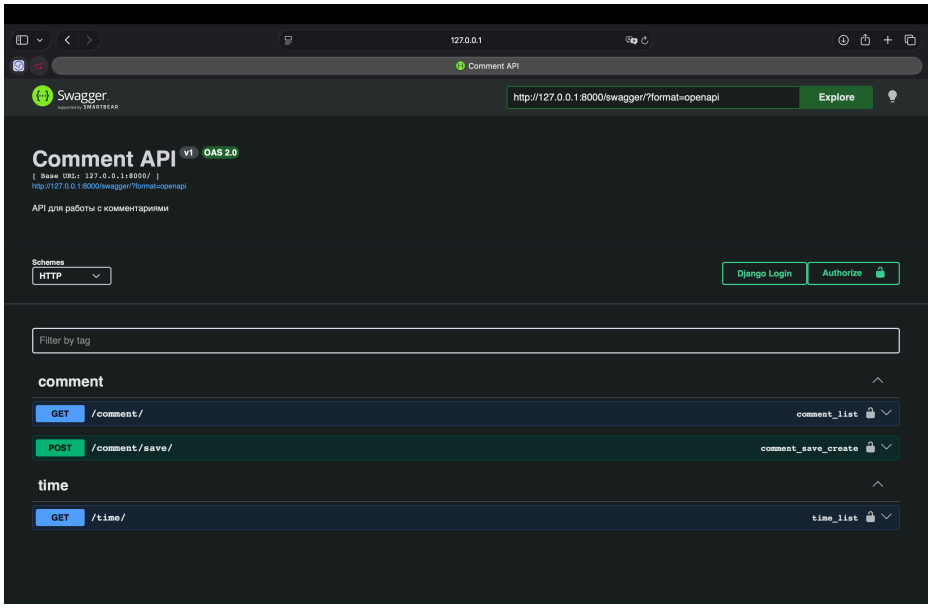


Рисунок 4 – Страница Swagger /swagger/

4. ВЫВОДЫ

В ходе лабораторной работы были изучены и на практике применены: архитектура RESTful API, библиотека Django REST Framework для создания API endpoints, сериализаторы для преобразования данных в JSON, асинхронное взаимодействие frontend и backend через Fetch API, автоматическое документирование API с помощью Swagger (drf-yasg). Создано полноценное приложение с моделью данных, REST endpoints и интерактивной HTML-страницей, работающей без перезагрузки. Поставленные задачи выполнены полностью.

5. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1: Что такое REST?

Ответ: REST (Representational State Transfer) – архитектурный стиль проектирования сетевых приложений, основанный на использовании стандартных HTTP-методов (GET, POST, PUT, DELETE) и безсостоянии взаимодействия между клиентом и сервером. Каждый ресурс идентифицируется URI, а операции соответствуют семантике HTTP-методов.

Вопрос 2: Формат данных JSON

Ответ: JSON (JavaScript Object Notation) – текстовый формат обмена данными, основанный на синтаксисе JavaScript. Представляет собой пары «ключ: значение» и массивы. Поддерживает строки, числа, булевы значения, null, объекты и массивы. Является стандартом де-факто для REST API благодаря лаконичности и нативной поддержке в JavaScript.

Вопрос 3: Какие методы не используются в REST?

Ответ: В классическом REST используются GET (получение), POST (создание), PUT (обновление), DELETE (удаление). Методы PATCH (частичное обновление)

и OPTIONS также применяются. Методы HEAD и TRACE используются редко. Нестандартные методы или расширения HTTP не относятся к REST.

Вопрос 4: Что такое swagger и его назначение?

Ответ: Swagger – это спецификация и набор инструментов для описания REST API. drf-yasg автоматически генерирует интерактивную документацию на основе кода Django, включая список endpoint'ов, параметры, форматы запросов и ответов, а также позволяет тестировать API прямо из браузера. Назначение – документирование и тестирование API без ручного написания документации.

Вопрос 5: За что отвечает протокол GraphQL?

Ответ: GraphQL – это протокол запросов к API, разработанный Facebook. В отличие от REST, где сервер определяет структуру ответа, в GraphQL клиент сам формирует запрос, указывая только необходимые поля. Это снижает объём передаваемых данных и количество запросов, устраняя проблему over-fetching и under-fetching.